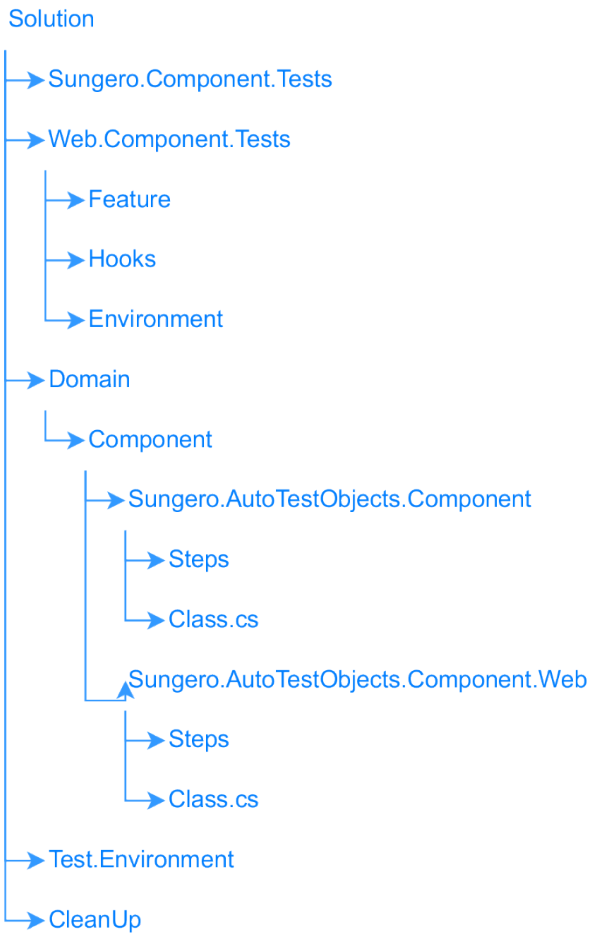
Соглашение по оформлению кода и сценариев в автотестах

Структура проекта (в разработке)

Схематично структура проекта выглядит так.


Solution
Sungero.Component.Tests
Web.Component.Tests
Feature
Hooks
Environment
Domain
Component
Sungero.AutoTestObjects.Component
Steps
Class.cs
Sungero.AutoTestObjects.Component.Web
Steps
Class.cs
Test.Environment
CleanUp

Определения:

- **Component** — название решения или области решения. Например, ESM — решение, Contract — область решения DirectumRX.

Описание схемы:

- **Sungero.Component.Tests** — Проект API-тестов на область Component. Например, Sungero.Vulnerability.Tests — тесты на уязвимости.\nМогут находить внутри папки, если проекты тестов можно объединить по какому-то признаку.
- **Web.Component.Tests** — Проект UI-тестов на область Component. Например, Web.Esm.Tests — тесты на решение ESM, Web.Contract.Tests — тесты на модуль Contract.\nМогут находить внутри папки, если проекты тестов можно объединить по какому-то признаку.
    - **Features**: Папка для .feature-файлов (например, CreateBusinessUnit.feature, ChangeDocumentType.feature).
    - **Hooks:** Папка для Hooks-файлов с предусловиями и постусловиями для сценариев.
    - **Environment**: Настройка окружения для проекта. Ссылка на существующие файлы настройки из Test.Environment.
- **Domain** — папка с классами и шагами, описывающие предметную область системы.
    - **Sungero.AutoTestObjects.Component:** Проект с описанием классов для работы с сущностями компоненты через API.\nМогут находить внутри папки, если классы можно объединить по какому-то признаку.
    - **Sungero.AutoTestObjects.Component.Web:** Проект с описанием классов для работы с сущностями компоненты на Web-странице.\nМогут находить внутри папки, если классы можно объединить по какому-то признаку.
        - **Class.cs**: Класс с описанием типа сущности.
        - **Steps**: Папка содержит классы с шагами (Steps), реализующими сценарии Gherkin (если необходимы).
- **Test.Environment** — папка с данными для настройки прогона (шаблоны документов, файлы с настройками окружения и параметрами запуска тестов).
- **CleanUp** — папка с методами очистки стендов после прогона автотестов.

Стиль кода

ВАЖНО! Всегда придерживайтесь правил чистого кода и принципов ООП.

- **Переменные**: CamelCase (userName).
- **Классы, константы, свойства и методы**: PascalCase (BusinessUnit.cs, ClerkLogin = "lena", DisplayName, GetBusinessUnitByName()).
- **Фигурные скобки**: На новой строке.

```
public void DoSomething()
{
    if (condition)
    {
```

```
            // Код
        }
    }
```

- **Названия классов и переменных:**
  - Содержательные, краткие, отражающие суть.
  - Начинаются с существительных.
- **Название методов**:
  - Отражающие суть метода.
  - Начинаются с глагола.
- **Константы**: Выносите повторяющиеся значения в константы.
- **using:** Должны оставаться только используемые сборки.

Документация

- **Комментарии**:
  - Не допускаются закомментированные строки кода. Код не нужен — удали.
  - Оставляйте комментарии для сложной логики.
  - Оставляйте комментарии с пояснением к UI-сценариям в Feature-файлах, у которых только API-проверки.
  - Избегайте избыточных комментариев, которые дублируют код.
- **README.txt**: Содержит информацию о дополнительных настройках системы, которые нельзя вынести в генератор, и о запуске тестов. Может содержать ссылки на статьи.

Обработка исключений

- Используйте try/catch только для ожидаемых ошибок. Непроверяемые исключения должны проваливать тест.

Feature-файлы

- Используйте CamelCase для именования feature-файлов (например, CreateBusinessUnit.feature);
- Feature-файл должен содержать набор схожих по смыслу сценариев;
- Название Feature-файлов должно отображать область, для которой написаны сценарии.
- В сценариях должен проверяться UI.
- Если в сценарии все проверки через API, то обязательно должен быть комментарий с пояснением зачем тест нужен в UI.

Оформление файлов

Структура сценария:

```
Feature: Название функциональности
  Краткое описание.
```

```
@Tags
@BeforeНазваниеСценария
@AfterНазваниеСценария
Scenario: Название сценария
  Given [Предусловие]
  And [Предусловие]
  When [Проверяемое действие]
  And [Проверяемое действие]
  Then [Ожидаемый результат]
  And [Ожидаемый результат]

Examples:
   |название параметра        |
   |первое значение параметра |
   |второе значение параметра |
```

Теги

Используйте теги для категоризации. Избегайте избыточных тегов.

Зарезервированные теги:

- Smoke — тест будет выполнен при проверке билда с продуктовой версией системы.
- Ignore — игнорировать тест при прогоне (тест не будет выполнен).
- CustomRetry(число) — количество повторов теста, если он упал. По умолчанию количество повторов — 2, даже если не указывать этот тег. Если тест выполнен успешно, то повторов не будет.

Пример:

```
@Smoke
@CRUD
@RunCoverAction
@BeforeСозданиеНашейОрганизации
@AfterСозданиеНашейОрганизации
Scenario Outline: Создание нашей организации
```

Оформление хуков (Before/After)

- **Имя хука:** Суффикс Before/After **+** название сценария на русском языке в PascalCase.
  - ***Исключение:*** имя [AfterScenario] может не совпадать с названием сценария если он используется в нескольких сценариях.
- **Один сценарий — один [BeforeScenario]**. Использование нескольких не допускается.
- **Можно использовать несколько [AfterScenario].**

Пример:

```
@BeforeКопированиеВложенийВПоручениеИзЗаданияНаРассмотрение
@AfterTestWithTwoCards
Scenario: Копирование вложений в поручение из задания на рассмотрение

@BeforeЗаполнениеКарточкиПодчиненногоПорученияПриСозданииИзКарточкиПростогоПоручения
@AfterTestWithTwoCards
Scenario: Заполнение карточки подчиненного поручения при создании из карточки простого
поручения

@BeforeДоступностьДействияСоздатьПорученивДиалогеПодтвержденияПриВыполнениизаданияНаРасс
```

```
мотрениеВРамкахСогласованияПоРегламенту
@AfterCloseDialog
@AfterSaveAndCloseCard
Scenario: Доступность действия Создать поручение в диалоге подтверждения при выполнении задания на рассмотрение в рамках согласования по регламенту

@BeforeИзменениеСвойствКарточкиОрганизации
@AfterИзменениеСвойствКарточкиОрганизации
Scenario: Изменение свойств карточки организации
```

Название сценария

- Строится по шаблону: **[Что проверяем][Объект проверки][Условия][Место]**
- Пишите на русском языке без орфографических ошибок;
- Избегайте избыточных слов (например, "Проверка" в начале).

Пример:

```
Scenario: Изменение свойств карточки организации

Scenario: Создание подразделения нашей организации

Scenario: Отправка подчиненного поручения, созданного из карточки простого поручения

Scenario: Заполнение карточки подчиненного поручения при создании из карточки простого поручения

Scenario: Переадресация задания сотруднику, уже участвующему в согласовании в рамках согласования по регламенту
```

Шаги Given/When

- Начинаются с **глагола в инфинитиве** (например, "Открыть", "Ввести", "Нажать").
- Четко описывают выполняемое в шаге действие.
- When ставится на шаге с которого начинается проверяемое действие.

Пример:

```
Scenario: Выполнение объединенного задания на подписание и рассмотрение с результатом Принято к сведению
       Given LoginPage. Войти в систему под учетной записью nata. Пароль 11111
       And NavigateTo. Открыть карточку задания на подписание и рассмотрение
       When AssignmentCard. Выполнить задание на подписание и рассмотрение с результатом:Принято к сведению
       Then BaseDocument. На служебной записке должна присутствовать утверждающая подпись сотрудника Natalie Ardo
       And ApprovalTask. Сотруднику Nadine Williams должно прийти задание на исполнение задания

Scenario: Вложение документа в простое поручения при создании из карточки документа
    Given LoginPage. Войти в систему под учетной записью nadya. Пароль 11111
    And NavigateTo. Открыть карточку входящего письма
    When OfficialDocumentCard. Отправить входящее письмо поручением
    And ActionItemCard. Заполнить параметры простого поручения
    And BaseCard. Сохранить карточку простого поручения
    And BaseCard. Получить ИД карточки
    Then Task. В области вложений входящий документ должен присутствовать
```

Шаги Then

- Содержат слово **"должен"** (или его формы: "должны", "должно").
- Описывают ожидаемый результат.
- Один Then — одна проверка.

Пример:

```
Scenario: Создание задачи на согласование по регламенту из карточки служебной записки
      Given LoginPage. Войти в систему под учетной записью zakonov. Пароль 11111
      And Explorer. Открыть папку Избранное. Ожидать загрузку:true
      And BaseGrid. Выбрать запись по значению в колонке по умолчанию
      When BaseGrid. Выполнить действие Отправить на согласование по регламенту в
контекстном меню
      Then BaseCard. Свойство Регламент должно заполниться Approval memo
      And BaseCard. Свойство Адресат должно заполниться George King
      And BaseCard. Свойство Обязательные согласующие должно заполниться Kate Philips
      And BaseCard. Карточка должна быть валидна

Scenario: Запрос продления срока согласования протокола в рамках регламента сотрудником
без руководителя
      Given LoginPage. Войти в систему под учетной записью irina. Пароль 11111
      And NavigateTo. Открыть карточку задания на согласование документа
      When ApprovalAssignmentCard. Запросить продление срока
      And Help. Сменить контекст карточки задания на согласование по регламенту на
карточку задачи на продление срока
      Then BaseCard. Карточка должна быть открыта
      And BaseCard. Свойство Кому должно заполниться значением Инициатор согласования
по регламенту
      And BaseCard. Свойство Кому должно быть доступно только на просмотр
```

Examples

- Содержит параметры для тестов. Например, способ открытия карточки.
- Содержит параметр **browser** (конфигурация клиентской машины) если тесты относятся к UI.
    - В параметре browser обязательно **должна быть конфигурация LocalChrome**, кроме тестов на DDS.
- Может отсутствовать, если тесты выполняются в API (например, тесты на отказоустойчивость системы).

Существующие конфигурации клиентских машин:

- LinuxFireFox — виртуальная машина под управлением ОС Linux с браузером Firefox;
- WinChrome — виртуальная машина под управлением ОС Windows с браузером Chrome;
- WinYandex — виртуальная машина под управлением ОС Windows с браузером Yandex;
- LocalChrome — локальная машина с браузером Chrome;
- WinDSDesktop — десктопное приложение нового DDS.

Пример:

```
Scenario: Доступность действий Выполнить и Создать поручение в задании на печать,
регистрацию и создание поручений
      Given LoginPage. Войти в систему под учетной записью nadya. Пароль 11111
      When NavigateTo. Открыть карточку задания на печать, регистрацию и создание
поручений
```

```
      Then BaseCard. Действия Выполнить и Создать поручение на риббоне должны
отображаться

Examples:
  | browser      |
  | LinuxFireFox |
  | WinChrome    |
  | WinYandex    |
  | LocalChrome  |

Scenario: Отправить входящий счет по договору на согласование по регламенту
      Given LoginPage. Войти в систему под учетной записью katya. Пароль 11111
      And Explorer. Открыть папку Избранное. Ожидать загрузку:false
      And BaseGrid. Открыть карточку входящего счета со значением в колонке Имя
      When OfficialDocumentCard. Отправить входящий счет на согласование по регламенту
      Then WorkflowCard. В карточке задачи должно быть вложение входящий счет
      And Help. Сменить контекст с входящего письма на договор
      And WorkflowCard. В карточке задачи должно быть вложение договор

Examples:
  | browser      | displayMode |
  | LinuxFireFox | Simple      |
  | LinuxFireFox | Satelite    |
  | WinChrome    | Simple      |
  | WinChrome    | Satelite    |
  | WinYandex    | Simple      |
  | WinYandex    | Satelite    |
  | LocalChrome  | Simple      |
  | LocalChrome  | Satelite    |

Scenario: Создание Решения
      Given Утилиты. Открыть прикладную разработку
      When Дерево решений. Добавить новое решение
      And Диалог создания элемента разработки. Заполнить Имя решения
      Then Дерево решений. Созданный элемент разработки должен отображаться в дереве
решений
      And Панель вкладок. Должна открыться вкладка с редактором решения
      And Редактор решения. Редактор решения должен открыться

Examples:
  | browser      |
  | WinDSDesktop |

Scenario: Работоспособность Storage после перезапуска сервиса
    Given Service. Остановить через Ansible Storage Service
    And Service. Storage сервис остановлен
    And Service. Запустить через Ansible Storage Service
    And Service. Storage сервис запущен
    When DocumentWithSplitter. Создать документ с телом
    Then BaseDocument. Документ должен сохраниться
    And BaseDocument. Количество версий должно быть равно: 1
```

Пример feature-файла

```
Feature: ConcatStages

Как пользователь,
```

```
хочу получать одно общее задание на несколько этапов, в которых я исполнитель, в рамках
согласования по регламенту,
чтобы не выполнять подряд несколько задании

      @BeforeДоступностьДействийВыполнитьИСоздатьПоручениеВЗаданииНаПечатьРегистрацию
озданиеПоручений
      @AfterScenarioCloseCard
      Scenario: Доступность действий Выполнить и Создать поручение в задании на печать,
регистрацию и создание поручений
            Given LoginPage. Войти в систему под учетной записью nadya. Пароль 11111
            When NavigateTo. Открыть карточку задания на печать, регистрацию и создание
поручений
            Then BaseCard. Действия Выполнить и Создать поручение на риббоне должны
отображаться

      Examples:
      | browser      |
      | LinuxFireFox |
      | WinChrome    |
      | WinYandex    |
      | LocalChrome  |

      #При выполнении задания из UI срабатывает клиентский код, в котором проверяется
объединение этапа с другими этапами
      #Тест проверяет срабатывание этой логики
      @BeforeВыполнениеОбъединенногоЗаданияНаПечатьРегистрацию
ИСозданиеПоручений
БезРегистрацииДокумента
      @AfterScenarioCloseCard
      Scenario: Выполнение объединенного задания на печать, регистрацию и создание
поручений без регистрации документа
            Given LoginPage. Войти в систему под учетной записью nadya. Пароль 11111
            And NavigateTo. Открыть карточку задания на печать, регистрацию и создание
поручений
            When AssignmentCard. Выполнить задание на печать, регистрацию и создание
поручений с результатом Выполнить без подтверждения
            Then BaseCard. Должно появиться сообщение валидации Для выполнения задания
нужно зарегистрировать документ. Тип сообщения Error

      Examples:
      | browser      |
      | LinuxFireFox |
      | WinChrome    |
      | WinYandex    |
      | LocalChrome  |
```

Hooks (предусловия и постусловия)

- **Именование класса**: Название feature-файла + суффикс Hooks (например, CreateBusinessUnitHooks.cs).
- **На каждый feature-файл свой Hooks.cs** в папке Hooks.
- **Наследование:** От класса **WebSteps, если UI-тесты**, от класса **Steps, если API-тесты**.

Оформление файлов

```
[Binding]
public class Hooks : WebSteps
{
  #region Поля и свойства
  #endregion

  #region Before

  [BeforeScenario]
  public void Setup()
  {
    using (new TestLogs(""))
    {
    }
  }

  #endregion

  #region After

  [AfterScenario]
  public void Teardown()
  {
    using (new TestLogs(""))
    {
    }
  }

  #endregion

  #region Steps

  [StepDefinition()]
  public void StepMethod()
  {
    using (new TestLogs(""))
    {
    }
  }

  #endregion

  #region Методы
  #endregion

  #region Конструктор

  public Hooks(ScenarioContext scContext) : base(scContext)
  {
  }

  #endregion
}
```

Поля и свойства

- Размещаются в регионе #region Поля и свойства.
- Выносятся общие для методов поля и свойства
- Могут отсутствовать в Hooks.

```
#region Поля и свойства

private OfficialDocument document;

private ActionItemExecutionTask actionItem;

private const string DocumentBodyFileName = "ForCreatingDocFromFile.doc";

#endregion
```

BeforeScenario

- Размещается в регионе #region Before.
- Обязательно содержит общее логирование подготовки данных.

```
[BeforeScenario("BeforeOpenActionItemListFromIncomingLetterCard")]
public void BeforeOpenActionItemListFromIncomingLetterCard()
{
  using (new TestLogs("Создание входящего письма и отправка вложением в поручение для
проверки открытия списка поручений из карточки"))
  {
    //создание сущностей через API
    //выполнение действий с сущностями через API
    //заполнение ScContext
  }
}
```

AfterScenario

- Размещается в регионе #region After.
- Включает общее логирование действий по очистке данных после теста.
- Обязательно содержит порядок выполнения Order = №. (0 — выполняется первым).
- Может отсутствовать в Hooks.

```
[AfterScenario("AfterCloseDialog", Order = 0)]
private void AfterClodeDialog()
{
  using (new TestLogs("Закрыть диалог подтверждения"))
  {
    //закрытие карточек, диалогов
    //удаление сущностей
  }
}
```

Step Definitions

- Размещаются в регионе #region Steps.
- Добавляются шаги, которые **применимы исключительно в сценариях, связанных с текущим Hooks**.

- Может иметь область Help, если шаг нужен только для смены записей в ScenarioContext. В остальных случаях область не указывается.
- Каждый шаг содержит логирование. Снятие скриншота обязательно, если шаг выполняется в UI (withScreen = true).
- Шаги помечаются **специфичными атрибутами Gherkin:** [StepDefinition], [Given], [When], [Then].
- Могут отсутствовать в Hooks.

```
[StepDefinition("Help. Сменить контекст на простое поручение")]
private void ChangeActionItemContext()
{
  using (new TestLogs("Сменить контекст с рассмотрения на простое поручение"))
  {
  }
}

[Then("Текст поручения должен совпадать с резолюцией адресата")]
private void CheckActionItemText()
{
  using (new TestLogs("Проверка совпадения текста поручения с резолюцией адресата"))
  {
  }
}
```

Методы

- Размещаются в регионе #region Методы.
- Выносятся вспомогательные методы для BeforeScenario, AfterScenario и StepDefinitions.
- Могут отсутствовать в Hooks.

Пример:

```
private Memo CreateMemo(string login)
{
  using (IntegrationClientConfigurator.Setup(login))
  {
      var memo = Memo.Create(DocumentBodyFileName, Constants.Manager);
      return memo;
  }
}
```

Конструктор

- Размещается в регионе #region Конструкторы.

Пример:

```
#region Конструктор

public OpenActionItemListHooks(ScenarioContext scContext) : base(scContext)
{
}

#endregion
```

Пример Hooks-файла

```
[Binding]
public class DeadlineExtensionHooks : WebSteps
{
  #region Поля и свойства

  Minutes minute;

  Task approvalTask;

  private const string DocumentBodyFileName = "ForCreatingDocFromFile.doc";

  #endregion

  #region Before


[BeforeScenario("BeforeЗапросПродленияСрокаСогласованияПротоколаВРамкахРегламентаСотрудн
икомБезРуководителя")]
  public void BeforeFillExtendDeadlineTaskCardFromRuleBasedApprovalAssignment()
  {
      using (new TestLogs("Отправка протокола задачей на согласование по регламенту"))
      {
        CreateMinuteAndSendByRuleBasedApprovalTask();
        var approvalAssignment =
this.approvalTask.GetJobs<ApprovalAssignment>().FirstOrDefault();

        ScContext[CardPageSteps.Context.Card] = new ApprovalAssignmentCard();
        ScContext[CardPageSteps.Context.SecondCard] = new ApprovalAssignmentCard();
        ScContext[EntitiesSteps.Context.Guid] = approvalAssignment.EntityGuid;
        ScContext[EntitiesSteps.Context.Id] = approvalAssignment.Id;
        ScContext[EntitiesSteps.Context.Value] = Constants.SecretaryUser;
        ScContext[CardPageSteps.Context.Property] =
DeadlineExtensionTask.PropertyNames.Assignee;
        }
  }

  #endregion

  #region Arter


[AfterScenario("AfterЗапросПродленияСрокаСогласованияПротоколаВРамкахСогласованияПоРегла
менту")]
  public void AfterFillExtendDeadlineTaskCardFromApprovalAssignment()
  {
      using (new TestLogs("Закрытие карточек"))
      {
        var firstCard = ScContext.Get<WorkflowCard>(CardPageSteps.Context.Card);
        var secondCard = ScContext.Get<WorkflowCard>(CardPageSteps.Context.SecondCard);

        if (firstCard != null && firstCard.CardExists())
            firstCard.CloseCard(true);
        if (secondCard != null && secondCard.CardExists())
            secondCard.CloseCard(true);
      }
  }
```

```
  #endregion

  #region Steps

  [When("Help. Сменить контекст карточки задания на согласование по регламенту на
карточку задачи на продление срока")]
  public void ChangeContextFromApprovalAssignmentToExtendDealineTask()
  {
    using (new TestLogs("Сменить контекст карточки на карточку задачи на продление
срока"))
      {
        var extendDeadlineTaskCard =
ScContext.Get<DeadlineExtensionTaskCard>(CardPageSteps.Context.Card);
        ScContext[EntitiesSteps.Context.Id] = extendDeadlineTaskCard.GetId();
        ScContext[EntitiesSteps.Context.EntityType] = extendDeadlineTaskCard.GetType();
      }
  }

  #endregion

  #region Методы

  private void CreateMinuteAndSendByRuleBasedApprovalTask()
  {
      using (IntegrationClientConfigurator.Setup(Constants.SecretaryLogin))
      {
        this.minute = Minutes.Create(DocumentBodyFileName,
DocumentKind.Entities.Protocols);
        var approvalTaskParams = new ApprovalTaskParams
        {
             Attachment = new Attachments(Minutes.AssemblyQualifiedName, this.minute),
             Rule =
ApprovalRule.GetActiveRuleByName(ApprovalRule.Entities.ApprovalProtocolCompanywithoutCEO
)
        };

        this.approvalTask = ApprovalTask.Create(approvalTaskParams);
        this.approvalTask.Start();
      }
  }

  #endregion

  #region Конструктор

  public DeadlineExtensionHooks(ScenarioContext scContext) : base(scContext)
  {
  }

  #endregion
}
```

Предметные классы

- Повторяют иерархию сущностей в платформе и решениях.

- Делятся на классы:
    - для работы с сущностью в сервисе интеграции
    - для работы с сущностью в вебе
    - шаги для Gherkin

API-классы

- **Расположение:** Domain → Решение → Sungero.AutoTestObjects.<Решение> → Модуль (если есть) → Класс. \nНапример, Domain → DirectumRX → Sungero.AutoTestObjectsRX → Company → BusinessUnit.cs.
- **Именование классов**: Совпадает с названием сущности в исходниках решения (например, BusinessUnit.cs).
- **Наследование:** Совпадает с иерархией классов в исходниках решения (например, BusinessUnit наследуется от Group).\nОсновной предок всех классов типов сущностей — Entity.\nОсновной предок модулей — BaseModule.
- **Назначение:** Работа с типом сущности, получение ресурсов и полей типа сущности через сервис интеграции.

Оформление файлов

```
 public class Class : ParentClass
  {
    #region Поля и свойства

    public new static readonly string CollectionName = "Название сущности в сервисе
интеграции";
    public override string Collection => CollectionName;

    private new static readonly string AssemblyQualifiedName = "Полное имя сборки";

    private static readonly Guid Guid = new Guid("");
    public override Guid EntityGuid => Guid;

    #endregion

    #region Методы

    #endregion

    #region Вложенные классы

    public new class LocalizedProperties : ParentClass.LocalizedProperties
    {

    }

    public new class PropertyNames : ParentClass.PropertyNames
    {

    }

    public new class Entities
    {
```

```
    }

    #endregion

    #region Конструкторы

    public Class(IDictionary<string, dynamic> data) => Data = data;

    public Class() => Data = new Dictionary<string, dynamic>();

    #endregion
  }
```

Поля и свойства

- Размещаются в регионе #region Поля и свойства.
- Классы, описывающие типы сущностей, содержат как минимум 3 свойства:
    - **Guid** — guid в который помещается идентификатор сущности.
    - **CollectionName** — название сущности в сервисе интеграции.
    - **AssemblyQualifiedName** — полное имя сборки, свойство необходимо для работы с ресурсами.
- Классы, описывающие модули, содержат как минимум одно свойство:
    - **Name** — Наименование модуля в сервисе интеграции.
- Содержит свойства и коллекции, которые относятся к типу сущности.

Методы

- Размещаются в регионе #region Методы.
- Реализуют логику работы с экземпляром типа сущности через сервис интеграции. Например, метод Create в BusinessUnit.cs создает запись Нашей организации через сервис интеграции.
- Могут отсутствовать.

Вложенные классы

- **Может содержать стандартные классы:**
    - PropertyNames — названия свойств типа сущности.
    - LocalizedProperties — локализованные названия свойств типа сущности.
    - Resources — ресурсы, относящиеся к типу сущности.
    - Entities — конкретные экземпляры типа сущности, созданные в системе и сохраненные в базе данных.
    - Actions — локализованные названия действий, добавленных в сущности.
    - CoverActions — локализованные названия действий на обложке модуля.
    - CoverTabs — локализованные названия вкладок на обложке модуля.

ВАЖНО: добавляйте только те вложенные классы, в которых есть необходимость.

- Можно добавлять не стандартные классы, если в этом есть необходимость.
- Могут отсутствовать.

Конструкторы

- Размещается в регионе #region Конструкторы.

Пример класса

```
  public class Agenda : InternalDocumentBase
  {
    #region Поля и свойства

    public new static readonly string CollectionName = "IAgendas";
    public override string Collection => CollectionName;

    private new static readonly string AssemblyQualifiedName =
"Sungero.Meetings.IAgenda, Sungero.Domain.Interfaces";

    public new static string DisplayName { get; } =
Resource.GetEntitySystemResource(AssemblyQualifiedName, "DisplayName");

    public new static string CollectionDisplayName { get; } =
      Resource.GetEntitySystemResource(AssemblyQualifiedName, "CollectionDisplayName");

    private static readonly Guid Guid = new Guid("5261da93-7879-4210-b3db-
c92fa894ab4d");
       public override Guid EntityGuid => Guid;

       private Meeting _meeting;
    public Meeting Meeting => _meeting
                              ?? (_meeting = new
Meeting(Data.GetEntityValue(PropertyNames.Meeting)));

    #endregion

    #region Вложенные классы

    public new class LocalizedProperties : InternalDocumentBase.LocalizedProperties
    {
      public static readonly string Meeting =
        Resource.GetEntitySystemResource(AssemblyQualifiedName, "Property_Meeting");
    }

    public new class PropertyNames : InternalDocumentBase.PropertyNames
    {
      public static readonly string Meeting = "Meeting";
    }

    #endregion

    #region Методы

    /// <summary>
    /// Создать повестку совещания.
    /// </summary>
    /// <param name="path">Путь до файла документа.</param>
```

```
    /// <param name="meeting">Совещания.</param>
    /// <param name="documentKind">Вид документа.</param>
    /// <returns>Повестка совещания.</returns>
    public static Agenda Create(string path, Meeting meeting, string documentKind)
    {
      var values = new Dictionary<string, object>
      {
        { BaseDocument.PropertyNames.Name, TestDataGenerator.GenerateString(15) },
        { PropertyNames.Meeting, meeting },
        { OfficialDocument.PropertyNames.DocumentKindName,
DocumentKind.Get(documentKind) }
      };
      var doc = Create<Agenda>(values);
      doc.CreateVersionFrom(path);
      doc = doc.Update<Agenda>();

      return doc;
    }

    #endregion

    #region Конструкторы

    public Agenda(IDictionary<string, dynamic> data) => Data = data;

    public Agenda() => Data = new Dictionary<string, dynamic>();

    #endregion
  }
```

Web-классы

- **Расположение:** Domain → Решение → Sungero.AutoTestObjects.<Решение>.Web → Модуль (если есть) → Класс. \nНапример, Domain → DirectumRX → Sungero.AutoTestObjectsRX.Web → Company → BusinessUnitCard.cs.
- **Именование классов**: Совпадает с названием сущности в исходниках решения + суффикс (например, BusinessUnitCard.cs).\nСуффиксы:
  - Card — класс описывает карточку;
  - Grid — класс описывает список;
  - WebModule — класс описывает обложку модуля;
  - Dialog — класс описывает диалог.
- **Наследование:** Совпадает с иерархией классов в исходниках решения (например, ContractCard наследуется от ContractualDocumentCard).\nОсновной предок всех классов типов сущностей — PageObject.\nОсновной предок модулей — BaseWebModule.
- **Назначение:** Работа с экземпляром типа сущности на Web-странице с помощью Selenium.

Оформление файлов

```
public class ClassCard: ParentClassCard
  {
    #region Поля и свойства
```

```
    protected override string caption => Class.DisplayName;

     protected override string EntityType => "ИД типа сущности";

    public override IDictionary<string, BaseControl> SimpleProperties
    {
      get
      {
        var result = base.SimpleProperties;
        return result;
      }
    }

    #endregion

    #region Методы

    #endregion

    #region Вложенные классы

    #endregion

    #region Конструкторы

    public ClassCard(string caption) : base(caption)
    {
    }

    public ClassCard() : base()
    {
    }

    #endregion
  }
```

Поля и свойства

- Размещаются в регионе #region Поля и свойства.
- Содержит как минимум 3 свойства (кроме классов, описывающих модуль):
  - **caption** — Отображаемое имя для типа сущности, в UI обычно отображается в левом верхнем углу карточки.
  - **EntityType** — Идентификатор типа сущности.
  - **SimpleProperties** — Словарь типизированных контролов свойств сущности. Каждый наследник дополняет словарь предка.
- Может содержать определение связанных элементов.
- Может содержать локаторы для поиска web-элементов на странице.

Методы

- Размещаются в регионе #region Методы.

- Реализуют логику работы с экземпляром типа сущности через взаимодействие с элементами на Web-странице. Например, метод OpenMeetingList в AgendaCard.cs открывает список совещаний по кнопке […] в карточке.
- Могут отсутствовать.

Вложенные классы

- **Вложенные классы:**
    - ActionsId — Идентификаторы добавленных в сущности действий в разметке страницы.

ВАЖНО: добавляйте только те вложенные классы, в которых есть необходимость.

- Можно добавлять не стандартные классы, если в этом есть необходимость.
- Могут отсутствовать.

Конструкторы

- Размещается в регионе #region Конструкторы.

Пример класса

```
public class AgendaCard : InternalDocumentBaseCard
  {
    #region Поля и свойства

    protected override string caption => Agenda.DisplayName;

    public override IDictionary<string, BaseControl> SimpleProperties
    {
      get
      {
        if (this.simpleProperties != null) return this.simpleProperties;
        var result = base.SimpleProperties;
        result[Agenda.PropertyNames.Meeting] = new NavigationControl(this.Card,
Agenda.LocalizedProperties.Meeting);
        return result;
      }
    }

    #endregion

    #region Методы

    public void OpenMeetingList()
    {
      this.OpenEntityCollection(Minutes.LocalizedProperties.Meeting);
      var modalGrid = new MeetingModalGrid(Meeting.CollectionDisplayName);
      this.WaitIsLoadData();
      if (!modalGrid.Exists())
        throw new WebGuiException("Модальный список с совещаниями не был открыт.");
      TestLogs.Screen();
    }

    #endregion

    #region Конструкторы
```

```
    public AgendaCard(string caption) : base(caption)
    {
    }

    public AgendaCard() : base()
    {
    }

    #endregion
  }
```

Steps-классы

- **Расположение:**
  - **Для UI-шагов**: Domain → Решение → Sungero.AutoTestObjects.<Решение>.Web → Модуль (если есть) → Steps → Класс. \nНапример, Domain → DirectumRX → Sungero.AutoTestObjectsRX.Web → Company → Steps → BusinessUnitCardSteps.cs.
  - **Для API-шагов:** Domain → Решение → Sungero.AutoTestObjects.<Решение> → Модуль (если есть) → Steps → Класс. \nНапример, Domain → DirectumRX → Sungero.AutoTestObjectsRX → Company → Steps → BusinessUnitSteps.cs.
- **Именование классов**: Совпадает с названием API-класса или UI-класса, к которому относится + суффикс Steps.\nНапример, BusinessUnitCardSteps.cs для UI-шагов или BusinessUnitSteps.cs — для API-шагов.
- **Наследование:** Для UI-шагов — WebSteps, для API-шагов — Steps.
- **Назначение:** Реализация шагов из Feature-файлов.

Оформление файлов

```
[Binding]
public class ClassSteps : WebSteps
{
  [StepDefinition("Область. Название шага")]
  public void StepMethod()
  {
    using (new TestLogs(""))
    {
    }
  }

  public class Context
  {
  }

  public BusinessUnitSteps(ScenarioContext scContext) : base(scContext)
  {
  }
}
```

Step Definitions

- Добавляются шаги, которые **применимы для разных сценариев тестов**.

- В области указывается контекст, в котором выполняется действие. \nНапример, в шаге "BusinessUnitCard. Заполнить код НОР значением (.*)" заполнение кода НОР вызывается в карточке BusinessUnit.
- Каждый шаг содержит логирование. Снятие скриншота обязательно, если шаг выполняется в UI (withScreen = true).
- Шаги помечаются **специфичными атрибутами Gherkin:** [StepDefinition], [Given], [When], [Then].

Методы

- Размещаются в регионе #region Методы.
- Выносятся вспомогательные методы для StepDefinitions.
- Могут отсутствовать.

Вложенный класс Context

- Размещается в конце файла.
- Содержит ключи для ScenarioContext.
- Ключи должны относиться к тому классу, в котором заданы.\nНапример, нельзя создать ключ "Договор" в классе BusinessUnitSteps, так как в карточке НОР нет договоров и работы с ними.
- Ключи классов предков можно использовать в классах наследниках.\nНапример, ключи из класса CardPageSteps можно использовать во всех карточках.
- Может отсутствовать.

Конструктор

- Размещается в регионе #region Конструкторы.

Пример Steps-файла

```
[Binding]
public class BusinessUnitCardSteps : WebSteps
{
  [StepDefinition("BusinessUnitCard. Заполнить код НОР значением (.*)")]
  public void SetCodeBusinessUnit(string code)
  {
    using (new TestLogs($"Заполнить код НОР значением {code}", true))
    {
      var card = ScContext.Get<BusinessUnitCard>(CardPageSteps.Context.Card);
      card.SetValue(BusinessUnit.PropertyNames.Code, code);
    }
  }

  public class Context
  {
        public static string Email = "Email";
  }

  public BusinessUnitSteps(ScenarioContext scContext) : base(scContext)
  {
  }
}
```